



시험에 나오는것만 공부한다!

시나공시리즈

기출문제

2024년 2회 정보처리기사 실기



정보처리기사 실기 시험은 한국산업인력공단에서 문제를 공개하지 않아 문제 복원에 많은 어려움이 있습니다. 다음에 제시된 문제는 시험을 치른 학생들의 기억을 토대로 복원한 것이므로, 일부 내용이나 문제별 배점이 실제 시험과 다를 수 있음을 알립니다.

저작권 안내

이 자료는 시나공 카페 회원을 대상으로 하는 자료로서 개인적인 용도로만 사용할 수 있습니다. 허락 없이 복제하거나 다른 매체에 옮겨 실을 수 없으며, 상업적 용도로 사용할 수 없습니다.

*** 수험자 유의사항 ***

1. 시험 문제지를 받는 즉시 응시하고자 하는 종목의 문제지가 맞는지를 확인하여야 합니다.
2. 시험 문제지 총면수·문제번호 순서·인쇄상태 등을 확인하고, 수험번호 및 성명을 답안지에 기재하여야 합니다.
3. 문제 및 답안(지), 채점기준은 일절 공개하지 않으며 자신이 작성한 답안, 문제 내용 등을 수험표 등에 이기(옮겨 적는 행위) 등은 관련 법 등에 의거 불이익 조치 될 수 있으니 유의하시기 바랍니다.
4. 수험자 인적사항 및 답안작성(계산식 포함)은 흑색 기구만 사용하되, 동일한 한 가지 색의 필기구만 사용해야 하며 흑색을 제외한 유색 필기구 또는 연필류를 사용하거나 2가지 이상의 색을 혼합 사용하였을 경우 그 문항은 0점 처리됩니다.
5. 답란(답안 기재란)에는 문제와 관련 없는 불필요한 낙서나 특이한 기록사항 등을 기재하여서는 안되며 부정의 목적으로 특이한 표식을 하였다고 판단될 경우에는 모든 문항이 0점 처리됩니다.
6. 답안을 정정할 때에는 반드시 정정부분을 두 줄(=)로 그어 표시하여야 하며, 두 줄로 긋지 않은 답안은 정정하지 않은 것으로 간주합니다.
7. 답안의 한글 또는 영문의 오타자는 오답으로 처리됩니다. 단, 답안에서 영문의 대·소문자 구분, 띄어쓰기는 여부에 관계 없이 채점합니다.
8. 계산 또는 디버깅 등 계산 연습이 필요한 경우는 <문 제> 아래의 연습란을 사용하시기 바라며, 연습란은 채점대상이 아닙니다.
9. 문제에서 요구한 가지 수(항수) 이상을 답란에 표기한 경우에는 답안기재 순으로 요구한 가지 수(항수)만 채점하고 한 항에 여러 가지를 기재하더라도 한 가지로 보며 그 중 정답과 오답이 함께 기재란에 있을 경우 오답으로 처리됩니다.
10. 한 문제에서 소문제로 파생되는 문제나, 가지수를 요구하는 문제는 대부분의 경우 부분채점을 적용합니다. 그러나 소문제로 파생되는 문제 내에서의 부분 배점은 적용하지 않습니다.
11. 답안은 문제의 마지막에 있는 답란에 작성하여야 합니다.
12. 부정 또는 불공정한 방법(시험문제 내용과 관련된 메모지사용 등)으로 시험을 치른 자는 부정행위자로 처리되어 당해 시험을 중지 또는 무효로 하고, 2년간 국가기술자격검정의 응시자격이 정지됩니다.
13. 시험위원이 시험 중 신분확인을 위하여 신분증과 수험표를 요구할 경우 반드시 제시하여야 합니다.
14. 시험 중에는 통신기기 및 전자기기(휴대용 전화기 등)를 지참하거나 사용할 수 없습니다.
15. 국가기술자격 시험문제는 일부 또는 전부가 저작권법상 보호되는 저작물이고, 저작권자는 한국산업인력공단입니다. 문제의 일부 또는 전부를 무단 복제, 배포, 출판, 전자출판 하는 등 저작권을 침해하는 일체의 행위를 금합니다.

※ 수험자 유의사항 미준수로 인한 채점상의 불이익은 수험자 본인에게 전적으로 책임이 있음

문제 1 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
public class Test {  
    public static void main(String[] args) {  
        String str = "ITISTESTSTRING";  
        String[] result = str.split("T");  
        System.out.print(result[3]);  
    }  
}
```

답 :

문제 2 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
public class Test {  
    public static void check(int[] x, int[] y)  
    {  
        if(x == y) System.out.print("O");  
        else System.out.print("N");  
    }  
  
    public static void main(String[] args) {  
        int a[] = new int[] {1, 2, 3, 4};  
        int b[] = new int[] {1, 2, 3, 4};  
        int c[] = new int[] {1, 2, 3};  
        check(a, b);  
        check(b, c);  
        check(a, c);  
    }  
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 3 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
def cnt(str, p):
    result = 0
    for i in range(len(str)):
        sub = str[i:i+len(p)]
        if sub == p:
            result += 1
    return result
str = "abdcabcabca"
p1 = "ca"
p2 = "ab"
print(f'ab{cnt(str, p1)} ca{cnt(str, p2)}')
```

답 :

문제 4 다음 설명에 해당하는 용어를 쓰시오. (5점)

- 시스템의 성능을 향상하고 개발 및 운영의 편의성 등을 높이기 위해 정규화된 데이터 모델을 의도적으로 통합, 중복, 분리하여 정규화 원칙을 위배하는 행위이다.
- 이를 수행하면 시스템의 성능이 향상되고 관리 효율성은 증가하지만 데이터의 일관성 및 정합성이 저하될 수 있다.
- 과도한 수행은 오히려 성능을 저하시킬 수 있다.

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 5 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main( ) {
    int a = 11;
    int b = 19;
    swap(a, b);
    switch(a) {
        case 1:
            b += 1;
        case 11:
            b += 2;
        default:
            b += 3;
            break;
    }
    printf("%d", a-b);
}
```

답 :

2024
시나공

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 6 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
void func(char *d, char *s) {
    while (*s) {
        *d = *s;
        d++;
        s++;
    }
    *d = '\0';
}

int main( ) {
    char* str1 = "first";
    char str2[50] = "teststring";
    int result = 0;
    func(str2, str1);
    for (int i = 0; str2[i] != '\0'; i++) {
        result += i;
    }
    printf("%d\n", result);
    return 0;
}
```

답 :

2024
시나공

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 7 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
interface Number {
    int sum(int[] a, boolean odd);
}

public class Test {
    public static void main(String[] args) {
        int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        OENumber OE = new OENumber( );
        System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false));
    }
}

class OENumber implements Number {
    public int sum(int[] a, boolean odd) {
        int result = 0;
        for (int i = 0; i < a.length; i++) {
            if ((odd && a[i] % 2 != 0) || (!odd && a[i] % 2 == 0)) {
                result += a[i];
            }
        }
        return result;
    }
}
```

답 :

2024
시나공

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 8 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
public class Test {
    public static String rf(String str, int index, boolean[] seen) {
        if(index < 0) return "";
        char c = str.charAt(index);
        String result = rf(str, index-1, seen);
        if(!seen[c]) {
            seen[c] = true;
            return c + result;
        }
        return result;
    }

    public static void main(String[] args) {
        String str = "abacabcd";
        int len = str.length( );
        boolean[] seen = new boolean[256];
        System.out.print(rf(str, len-1, seen));
    }
}
```

답 :

문제 9 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
#include <stdio.h>

int main( ) {
    int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    int* parr[2] = {arr[1], arr[2]};
    printf("%d", parr[1][1] + *(parr[1]+2) + **parr);
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 10 다음에 제시된 <학생>과 <학과> 테이블에 대한 릴레이션 스키마를 참고하여 각 SQL문의 요구사항에 맞도록 괄호(①~④)에 알맞은 답을 적어 SQL문을 완성하시오. (5점)

릴레이션 스키마

<학생>	학번(PK)	이름	나이	학과
<학부>	학번(PK)	이름	주소	나이

<SQL 1> 문

[요구사항] 신입생 정보가 확인되어 <학부> 테이블에 신입생 정보를 추가한다.

[SQL]

INSERT INTO 학부(학번, 이름, 주소, 나이) (①) (240912, '최재균', '서울', 20);

<SQL 2> 문

[요구사항] <SQL 1> 문에서 추가한 학생을 <학부> 테이블에서 검색한 후 검색된 자료를 <학생> 테이블에 추가한다.

[SQL]

INSERT INTO 학생(학번, 이름, 나이, 학과) (②) 학번, 이름, 나이, '컴퓨터공학' FROM 학부 WHERE 이름 = '최재균';

<SQL 3> 문

[요구사항] <학생> 테이블의 모든 자료를 조회한다.

[SQL]

SELECT * (③) 학생;

<SQL 4> 문

[요구사항] 학생이 휴학 신청해서 해당 학생의 '학과' 필드의 값을 "휴학"으로 변경한다.

[SQL]

UPDATE 학생 (④) 학과 = '휴학' WHERE 학번 = 240912;

답

- ①
- ②
- ③
- ④

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 11 다음 <회원> 테이블에서 카디널리티(Cardinality)와 디그리(Degree)를 구하십시오. (5점)

<회원>

ID	이름	거주지	신청강의
191-SR05	강순동	마포구	E01
024-RU09	김은경	관악구	S03
181-SQ03	이지연	서대문구	E02
059-RL08	윤경	광진구	S03
029-SX07	김민재	서대문구	E02

답

- ① 카디널리티(Cardinality) :
- ② 디그리(Degree) :

문제 12 다음 설명에 해당하는 프로토콜을 쓰시오. (5점)

- 네트워크 계층에서 IP 패킷 단위의 데이터 변조 방지 및 은닉 기능을 제공하는 프로토콜이다.
- 주요 구성 요소에는 AH(Authentication Header), ESP(Encapsulating Security Payload), SA(Security Association), IKE(Internet Key Exchange)가 있다.
- 주요 기능에는 암호화, 무결성, 인증, 재전송 방지가 있다.

답 :

문제 13 다음 설명에 해당하는 보안 알고리즘을 쓰시오. (5점)

- 2001년 미국 표준 기술 연구소(NIST)에서 발표한 개인키 암호화 알고리즘이다.
- DES의 한계를 느낀 NIST에서 공모한 후 발표하였다.
- 블록 크기는 128비트이며, 키 길이에 따라 명칭 뒤에 128, 192, 256을 붙여 구분한다.

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 14 네트워크에 관련된 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. (5점)

- (①) : 연결형 통신에서 주로 사용되는 방식으로, 출발지와 목적지의 전송 경로를 미리 연결하여 논리적으로 고정한 후 통신하는 방식
- (②) : 비연결형 통신에서 주로 사용되는 방식으로, 사전에 접속 절차를 수행하지 않고 헤더에 출발지에서 목적지까지의 경로 지정을 위한 충분한 정보를 붙여서 개별적으로 전달하는 방식

답

- ①
- ②

문제 15 모듈에 대한 다음 설명에 해당하는 응집도(Cohesion)를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. (5점)

모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도

<보기>

- | | | | |
|-----------|-----------|-----------|-----------|
| ㉠ 기능적 응집도 | ㉡ 순차적 응집도 | ㉢ 교환적 응집도 | ㉤ 절차적 응집도 |
| ㉥ 시간적 응집도 | ㉦ 논리적 응집도 | ㉧ 우연적 응집도 | |

답 :

문제 16 준비상태 큐에 각 프로세스의 도착 시간과 실행 시간이 다음과 같을 때 SRT(Shortest Remaining Time)로 스케줄링 할 때 평균 대기 시간을 쓰시오. (5점)

프로세스	도착 시간	실행 시간
A	0	8
B	1	4
C	2	9
D	3	5

답 :

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

연 습 란

문제 17 다음 설명에 해당하는 디자인 패턴을 <보기>에서 찾아 쓰시오. (5점)

- 자료 구조와 같이 접근이 잦은 객체에 대해 동일한 인터페이스를 사용하도록 하는 패턴이다.
- 내부 표현 방법의 노출 없이 순차적인 접근이 가능하다.

<보기>

생성 패턴	구조 패턴	행위 패턴
Abstract Factory Builder Factory Method Prototype Singleton	Adapter Bridge Composite Decorator Proxy	Command Interpreter Iterator Mediator Observer

답 :

문제 18 모듈에 대한 다음 설명에 해당하는 결합도(Coupling)를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. (5점)

- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도이다.
- 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 된다.

<보기>

㉠ 자료 결합도	㉡ 스템프 결합도	㉢ 제어 결합도
㉣ 공통 결합도	㉤ 내용 결합도	㉥ 외부 결합도

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

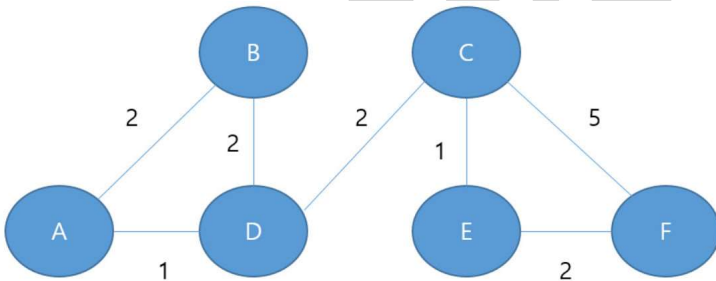
문제 19 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (5점)

```
#include <stdio.h>
struct node {
    int data;
    struct node *Next;
};

int main( ) {
    struct node *head = NULL;
    struct node a = {10, 0};
    struct node b = {20, 0};
    struct node c = {30, 0};
    head = &a;
    a.Next = &b;
    b.Next = &c;
    printf("%d", head -> Next -> data);
}
```

답 :

문제 20 다음 그림에서 ㉠~㉦는 라우터이고, 각 링크 상의 숫자는 가중치 값을 나타낼 때 RIP(Routing Information Protocol)을 이용한 A에서 F까지의 최적 경로를 쓰시오. (5점)



답 :

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

기출문제 정답 및 해설

[문제 1]

S

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 소문자 **s**로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

```
public class Test {  
    public static void main(String[] args) {  
        ❶ String str = "ITISTESTSTRING";  
        ❷ String[] result = str.split("T");  
        ❸ System.out.print(result[3]);  
    }  
}
```

- ❶ 문자열 객체 str을 선언하고, "ITISTESTSTRING"으로 초기화한다.
❷ 문자열 배열 result를 선언하고, str에 저장된 문자열을 'T'를 기준으로 분리하여 각각의 요소를 초기화한다. 배열을 선언할 때 배열의 크기를 지정하지 않으면 초기값의 크기로 배열의 요소가 만들어진다.
• A.split("분리문자") : A 문자열을 '분리문자'를 기준으로 분리하여 반환함, '분리문자'를 생략하면 공백을 기준으로 문자열을 분리함

	[0]	[1]	[2]	[3]	[4]
result	I	IS	ES	S	RING

- ❸ result[3]의 값을 출력한다.

결과 S

[문제 2]

NNN

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 소문자 **nnn**으로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

```

public class Test {
    ⑤⑧⑩ public static void check(int[] x, int[] y)
    {
        ⑥⑨⑫ if(x == y) System.out.print("O");
            else System.out.print("N");
    }
    public static void main(String[] args) {
        ① int a[] = new int[] {1, 2, 3, 4};
        ② int b[] = new int[] {1, 2, 3, 4};
        ③ int c[] = new int[] {1, 2, 3};
        ④ check(a, b);
        ⑦ check(b, c);
        ⑩ check(a, c);
    }
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

① 정수형 배열 a를 선언하고 초기화한다.

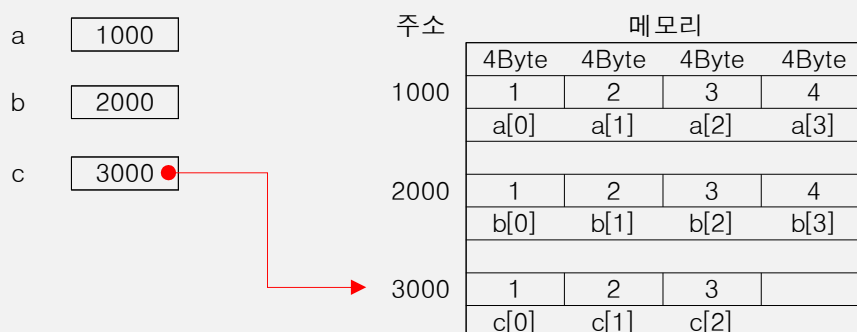
(이후 그림에서 a, b, c가 저장된 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했습니다.)



② 정수형 배열 b를 선언하고 초기화한다.



③ 정수형 배열 c를 선언하고 초기화한다.



④ a와 b를 인수로 check() 메소드를 호출한다. ⑤번으로 이동한다.

※ 인수로 배열의 이름을 지정하면 배열의 시작 주소가 인수로 전달됩니다.

⑤ 반환값이 없는 check() 메소드의 시작점이다. ④번에서 전달한 a와 b 배열의 주소를 x와 y가 받는다.

⑥ x와 y가 같으면 "O"를 출력하고, 다르면 "N"을 출력한다. x와 y는 값과 관계없이 주소가 다르므로 서로 다른 객체이다. "N"을 출력하고 if문이 종료되어, check()를 호출했던 ④번 아래 문장으로 제어를 옮긴다.

결과 **N**

⑦ b와 c를 인수로 check() 메소드를 호출한다. ⑧번으로 이동한다.

⑧ 반환값이 없는 check() 메소드의 시작점이다. ⑦번에서 전달한 b와 c 배열의 주소를 x와 y가 받는다.

⑨ x와 y는 서로 다른 객체이므로 “N”을 출력한다. if문이 종료되어 check()를 호출했던 ⑦번 아래 문장으로 제어를 옮긴다.

결과 **NN**

⑩ a와 c를 인수로 check() 메소드를 호출한다. ⑪번으로 이동한다.

⑪ 반환값이 없는 check() 메소드의 시작점이다. ⑩번에서 전달한 a와 c 배열의 주소를 x와 y가 받는다.

⑫ x와 y는 서로 다른 객체이므로 “N”을 출력한다. if문이 종료되어 check()를 호출했던 ⑩번 아래 문장으로 제어를 옮겨 프로그램을 종료한다.

결과 **NNN**

[문제 3]

ab3 ca3

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 대문자 **AB3 CA3**으로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

```
⑤⑬ def cnt(str, p):
⑥⑭     result = 0;
⑦⑮     for i in range(len(str)):
⑧⑯         sub = str[i:i+len(p)]
⑨⑰         if sub == p:
⑩⑱             result += 1
⑪⑲     return result
① str = "abdcabcbca"
② p1 = "ca"
③ p2 = "ab"
④⑫⑳ print(f'ab{cnt(str, p1)} ca{cnt(str, p2)}')
```

cnt() 메소드 정의부의 다음 줄인 8번째 줄부터 실행한다.

① 변수 str을 선언하고, “abdcabcbca”로 초기화한다.

② 변수 p1을 선언하고, “ca”로 초기화한다.

③ 변수 p2를 선언하고, “ab”로 초기화한다.

④ ab를 출력하고 str과 p1을 인수로 cnt() 메소드를 호출한다. ⑤번으로 이동한다.

※ **f-string 포맷** : f‘문자열{표현식}’의 형식으로, 예약어 f 뒤의 작은따옴표 안의 문자열은 그대로 출력하고 중괄호 안의 표현식은 그 결과 값을 출력한다.

결과 **ab**

⑤ cnt() 메소드의 시작점이다. ④번에 전달한 str과 p1을 str과 p가 받는다.

⑥ 변수 result를 선언하고, 0으로 초기화한다.

⑦ i에 0부터 str 변수의 길이인 11보다 1작은 10까지의 숫자를 순서대로 저장하며, ⑧~⑩번을 반복 수행한다.

※ len()은 문자열이나 배열의 길이를 반환한다.

⑧ 변수 sub를 선언하고, str의 i 번째부터 i에 ‘p의 길이인 2를 더한 것’ - 1 만큼, 즉 i 번째부터 2개의 문자를 추출해 저장한다.

※ **객체명[초기위치:최종위치]** : 객체의 ‘초기위치’에서 ‘최종위치’-1까지의 요소를 반환함

⑨ sub와 p가 같으면 ⑩번을 수행하고, 아니면 반복문의 시작인 ⑦번으로 이동한다.

⑩ ‘result = result + 1;’과 동일하다. result의 값을 1씩 누적시킨다.

반복문 실행에 따른 변수들의 변화는 다음과 같다.

cnt() 메소드																										
str											p	i	sub	result	반환값											
012345678910 <table><tr><td>a</td><td>b</td><td>d</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td><td>b</td><td>c</td><td>a</td></tr></table>											a	b	d	c	a	b	c	a	b	c	a	ca	0	ab	1	3
											a	b	d	c	a	b	c	a	b	c	a					
											1	bd														
											2	dc														
											3	ca														
											4	ab	2													
											5	bc														
											6	ca														
											7	ab														
											8	bc	3													
											9	ca														
10	a																									

- ⑪ result 값 3을 cnt()를 호출했던 ⑫번으로 반환한다.
- ⑫ ⑪번으로부터 반환받은 값 3을 출력하고 한 칸을 띄운 후 **ca**를 출력한다. 이어서 str과 p2를 인수로 cnt() 메소드를 호출한다. ⑬번으로 이동한다.
- 결과 **ab3 ca**
- ⑬ cnt() 메소드의 시작점이다. ⑫번에 전달한 str과 p2를 str과 p가 받는다.
- ⑭ 변수 result를 선언하고, 0으로 초기화한다.
- ⑮ i에 0부터 str 변수의 길이인 11보다 1 작은 10까지의 숫자를 순서대로 저장하며, ⑮~⑰번을 반복 수행한다.
- ⑯ 변수 sub를 선언하고, str의 i 번째부터 2개의 문자를 추출해 저장한다.
- ⑰ sub와 p가 같으면 ⑰번을 수행하고, 아니면 반복문의 시작인 ⑮번으로 이동한다.
- ⑱ 'result = result + 1;'과 동일하다. result의 값을 1씩 누적시킨다.
- 반복문 실행에 따른 변수들의 변화는 다음과 같다.

cnt() 메소드															
str											p	i	sub	result	반환값
012345678910 a b d c a b c a b c a											ab	0	ab	1	2
												1	bd		
												2	dc		
												3	ca	3	
												4	ab		
												5	bc		
												6	ca		
												7	ab	3	
												8	bc		
												9	ca		
												10	a		

- ⑲ result 값 3을 cnt()를 호출했던 ⑳번으로 반환한다.
- ㉑ ⑲번으로부터 반환받은 값 3을 출력한다.

결과 **ab3 ca3**

[문제 4]

※ 다음 중 하나를 쓰면 됩니다.

반정규화, Denormalization, 비정규화, 역정규화

[문제 5]

-13

[해설]

```
#include <stdio.h>
④ void swap(int a, int b) {
⑤     int t = a;
⑥     a = b;
⑦     b = t;
}

int main( ) {
①     int a = 11;
②     int b = 19;
③     swap(a, b);
⑧     switch(a) {
        case 1:
            b += 1;
⑨     case 11:
⑩         b += 2;
⑪     default:
⑫         b += 3;
⑬         break;
    }
⑭     printf("%d", a-b);
}
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

- ① 정수형 변수 a를 선언하고, 11로 초기화한다.
- ② 정수형 변수 b를 선언하고, 19로 초기화한다.
- ③ a와 b를 인수로 swap() 함수를 호출한다. ④번으로 이동한다.
- ④ 반환값이 없는 swap() 함수의 시작점이다. ③번에서 전달한 a와 b를 a와 b가 받는다.
- ⑤ ~ ⑦ 임시 변수 t를 사용하여 a와 b의 값을 교환하는 과정이다.

※ swap() 함수를 호출할 때 a, b 변수의 주소를 전달한 것이 아니므로 swap() 함수에서 a, b의 교환은 main() 함수의 a, b 변수에 영향을 주지 않습니다.

main() 함수		swap() 함수		
a	b	a	b	t
11	19	19	11	11

- ⑧ a의 값 11에 해당하는 숫자를 찾아간다. ⑨번으로 이동한다.
- ⑨ a가 11일 경우 찾아오는 곳이다. ⑩번을 실행한다.
- ⑩ 'b = b + 2;'와 동일하다. b의 값에 2를 더한다. b에는 21이 저장된다. break문이 없으므로, break문을 만날 때까지 모든 문장이 실행된다. ⑪번으로 이동한다.
- ⑪ a의 값에 해당하는 case문이 없는 경우 찾아오는 곳이지만 ⑩번 문장 수행 후 break문이 없어 실행되는 문장이다. ⑫번을 실행한다.
- ⑫ 'b = b + 3;'과 동일하다. b의 값에 3을 더한다. b에는 24가 저장된다.
- ⑬ switch문을 벗어나 ⑭번으로 이동한다.
- ⑭ 11-24의 결과를 정수형으로 출력한다.

결과 -13

[문제 6]

10

[해설]

```
#include <stdio.h>
❺ void func(char *d, char *s) {
❻       while (*s) {
❼           *d = *s;
❸           d++;
❹           s++;
        }
❿       *d = '\0';
    }

    int main( ) {
❶       char* str1 = "first";
❷       char str2[50] = "teststring";
❸       int result = 0;
❹       func(str2, str1);
❺       for (int i = 0; str2[i] != '\0'; i++) {
❻           result += i;
        }
❸       printf("%d\n", result);
❹       return 0;
    }
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

- ❶ 문자형 포인터 변수 str1을 선언하고, “first”가 저장된 주소로 초기화한다.
- ❷ 50개의 요소를 갖는 문자형 배열 str2를 선언하고, “teststring”가 저장된 주소로 초기화한다.
(이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했습니다.)

주소		메모리										
str1	1000	'f'	'i'	'r'	's'	't'	'\0'					
str2	2000	't'	'e'	's'	't'	's'	't'	'r'	'i'	'n'	'g'	'\0'

- ❸ 정수형 변수 result를 선언하고, 0으로 초기화한다.
- ❹ str2와 str1을 인수로 func() 함수를 호출한다. ❺번으로 이동한다.
※ 인수로 포인터 변수나 배열의 이름을 지정하면 포인터 변수나 배열의 시작 주소가 인수로 전달됩니다.
- ❺ 반환값이 없는 func() 함수의 시작점이다. ❹번에서 전달한 str2와 str1을 문자형 포인터 변수 d와 s가 받는다.

[문제 7]

25, 20

※ **답안 작성 시 주의 사항** : 프로그램의 실행 결과는 부분 점수가 없으므로 정확하게 작성해야 합니다. 예를 들어, 출력값 사이에 공백이나 콤마 없이 **25,20**나 **25 20**으로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

```

① interface Number {
②     int sum(int[] a, boolean odd);
}

public class Test {
    public static void main(String[] args) {
①         int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
②         OENumber OE = new OENumber( );
③④⑤     System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false));
    }
}

⑥ class OENumber implements Number {
⑦⑧⑨ public int sum(int[] a, boolean odd) {
⑩⑪     int result = 0;
⑫⑬     for (int i = 0; i < a.length; i++) {
⑭⑮         if ((odd && a[i] % 2 != 0) || (!odd && a[i] % 2 == 0)) {
⑯⑰             result += a[i];
        }
    }
⑱⑲     return result;
}
}

```

① 인터페이스 Number를 선언한다.

② 정수를 반환하는 sum()을 선언한다. 인터페이스에 선언된 메소드는 선언만 있고 내부에 실행 코드가 없는 추상 메소드이므로, 이후 상속 관계가 설정된 자식 클래스에서 재정의한 후 사용한다.

③ Number 인터페이스를 상속받는 클래스 OENumber를 정의한다.

④ 정수를 반환하는 sum()을 정의한다. sum() 메소드는 ②에서 선언된 추상 메소드(②)를 재정의하는 것이다.

※ 추상 메소드는 부모 클래스가 자식 클래스에게 주는 의무와 같습니다. 부모 클래스와 상속 관계에 있다면 반드시 부모 클래스의 추상 메소드를 재정의해야 합니다. 그렇지 않으면 오류가 발생합니다.

모든 Java 프로그램 반드시 main() 메소드에서 시작한다.

① 정수형 배열 a를 선언하고 초기화한다. 배열을 선언할 때 배열의 크기를 지정하지 않으면, 초기값의 개수로 배열의 크기가 결정된다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
a	1	2	3	4	5	6	7	8	9

② 클래스 OENumber의 객체 변수 OE를 선언한다.

③ a와 논리값 true를 인수로 OE의 sum() 메소드를 호출한 후 돌려받은 값을 출력한다. ④번으로 이동한다.

※ 인수로 배열의 이름을 지정하면 배열의 시작 주소가 전달됩니다.

④ 정수를 반환하는 sum() 메소드의 시작점이다. ⑤번에서 전달한 a 배열의 시작 주소와 논리값 true를 a와 odd가 받는다.

- ⑤ 정수형 변수 result를 선언하고, 0으로 초기화한다.
- ⑥ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 a 배열의 길이인 9보다 작은 동안 ⑦, ⑧번을 반복 수행한다.
- length : 배열 클래스의 속성으로, 배열 요소의 개수가 저장되어 있음
- ⑦ odd가 참(true)이고 a[i]를 2로 나눈 나머지가 0이 아니거나 odd가 거짓(false)이고 a[i]를 2로 나눈 나머지가 0이면 ⑧번을 수행하고 아니면 반복문의 시작인 ⑥번으로 이동한다.
- ⑧ 'result = result + a[i];'와 같다. result에 a[i]의 값을 누적시킨다.

반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	odd	a[i]	a[i] % 2	result
0	true	1	1	0
1		2	0	1
2		3	1	4
3		4	0	
4		5	1	9
5		6	0	
6		7	1	16
7		8	0	
8		9	1	25
9				

- ⑨ 함수를 호출했던 ⑩번으로 result의 값 25를 반환한다.
- ⑩ ⑨번으로부터 반환받은 값 25를 출력하고 콤마와 공백 한 칸을 출력한다. 이어서 a와 논리값 false를 인수로 OE의 sum() 메소드를 호출한 후 돌려받은 값을 출력한다. ⑪번으로 이동한다.

결과 25,

- ⑪ 정수를 반환하는 sum() 메소드의 시작점이다. ⑩번에서 전달한 a 배열의 시작 주소와 논리값 false를 a와 odd가 받는다.
- ⑫ 정수형 변수 result를 선언하고, 0으로 초기화한다.
- ⑬ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 a 배열의 길이인 9보다 작은 동안 ⑭, ⑮번을 반복 수행한다.
- ⑭ odd가 참(true)이고 a[i]를 2로 나눈 나머지가 0이 아니거나 odd가 거짓(false)이고 a[i]를 2로 나눈 나머지가 0이면 ⑮번을 수행하고 아니면 반복문의 시작인 ⑬번으로 이동한다.
- ⑮ 'result = result + a[i];'와 같다. result에 a[i]의 값을 누적시킨다.

반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	odd	a[i]	a[i] % 2	result
0	false	1	1	0
1		2	0	2
2		3	1	
3		4	0	6
4		5	1	
5		6	0	12
6		7	1	
7		8	0	20
8		9	1	
9				

- ⑯ 함수를 호출했던 ⑰번으로 result의 값 20을 반환한다.
- ⑰ ⑯번으로부터 반환받은 값 20을 출력한 후 프로그램을 종료한다.

결과 25, 20

[문제 8]

dcba

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 대문자 **DCBA**로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

```
public static void main(String[] args) {
  ❶ String str = "abacabcd";
  ❷ int len = str.length( );
  ❸ boolean[] seen = new boolean[256];
  ❹ System.out.print(rf(str, len-1, seen));
}
```

모든 Java 프로그램 반드시 main() 메소드에서 시작한다.

❶ 문자열 객체 str을 선언하고, "abacabcd"로 초기화한다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
str	'a'	'b'	'a'	'c'	'a'	'b'	'c'	'd'

❷ 정수형 변수 len을 선언하고 str의 길이인 8로 초기화한다.

• length : 배열 클래스의 속성으로 배열 요소의 개수가 저장되어 있음

❸ 256개의 요소를 갖는 논리형 배열 seen을 선언한다. 논리형 배열의 요소에는 true나 false 중 하나를 저장할 수 있으며, 기본값은 false이다.

	[0]	[1]	[2]	[3]	[4]	...	[253]	[254]	[255]
seen	false	false	false	false	false	...	false	false	false

❹ rf(str, 7, seen) 메소드를 호출한다.

※ 인수로 배열의 이름을 지정하면 배열의 시작 주소가 전달됩니다.

```
❺ public static String rf(String str, int index, boolean[] seen) { // index = 7, c = 'd'
❻ if(index < 0) return "";
❼ char c = str.charAt(index);
❽ String result = rf(str, index-1, seen);
```

❺ 문자열을 반환하는 rf() 메소드의 시작점이다. ❹번에서 전달한 str, 7, seen을 str, index, seen이 받는다.

❻ index가 0보다 작으면 빈 문자열을 반환한다. index가 7이므로 다음 문장을 수행한다.

❼ 문자형 변수 c를 선언하고, str 문자열에서 index 위치에 있는 문자로 초기화한다. index가 7이므로 c에는 'd'가 저장된다.

❽ 문자열 변수 result를 선언하고, rf(str, 6, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```
❾ public static String rf(String str, int index, boolean[] seen) { // index = 6, c = 'c'
❿ if(index < 0) return "";
⓫ char c = str.charAt(index);
⓬ String result = rf(str, index-1, seen);
```

❾ 문자열을 반환하는 rf() 메소드의 시작점이다. ❸번에서 전달한 str, 6, seen을 str, index, seen이 받는다.

❿ index가 6이므로 다음 문장을 수행한다.

⓫ 문자형 변수 c를 선언하고, str[6]의 값 'c'로 초기화한다.

⓬ 문자열 변수 result를 선언하고, rf(str, 5, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```

13 public static String rf(String str, int index, boolean[] seen) { // index = 5, c = 'b'
14     if(index < 0) return "";
15     char c = str.charAt(index);
16     String result = rf(str, index-1, seen);

```

- 13 문자열을 반환하는 rf() 메소드의 시작점이다. 12번에서 전달한 str, 5, seen을 str, index, seen이 받는다.
 14 index가 5이므로 다음 문장을 수행한다.
 15 문자형 변수 c를 선언하고, str[5]의 값 'b'로 초기화한다.
 16 문자열 변수 result를 선언하고, rf(str, 4, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```

17 public static String rf(String str, int index, boolean[] seen) { // index = 4, c = 'a'
18     if(index < 0) return "";
19     char c = str.charAt(index);
20     String result = rf(str, index-1, seen);

```

- 17 문자열을 반환하는 rf() 메소드의 시작점이다. 16번에서 전달한 str, 4, seen을 str, index, seen이 받는다.
 18 index가 4이므로 다음 문장을 수행한다.
 19 문자형 변수 c를 선언하고, str[4]의 값 'a'로 초기화한다.
 20 문자열 변수 result를 선언하고, rf(str, 3, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```

21 public static String rf(String str, int index, boolean[] seen) { // index = 3, c = 'c'
22     if(index < 0) return "";
23     char c = str.charAt(index);
24     String result = rf(str, index-1, seen);

```

- 21 문자열을 반환하는 rf() 메소드의 시작점이다. 20번에서 전달한 str, 3, seen을 str, index, seen이 받는다.
 22 index가 3이므로 다음 문장을 수행한다.
 23 문자형 변수 c를 선언하고, str[3]의 값 'c'로 초기화한다.
 24 문자열 변수 result를 선언하고, rf(str, 2, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```

25 public static String rf(String str, int index, boolean[] seen) { // index = 2, c = 'a'
26     if(index < 0) return "";
27     char c = str.charAt(index);
28     String result = rf(str, index-1, seen);

```

- 25 문자열을 반환하는 rf() 메소드의 시작점이다. 24번에서 전달한 str, 2, seen을 str, index, seen이 받는다.
 26 index가 2이므로 다음 문장을 수행한다.
 27 문자형 변수 c를 선언하고, str[2]의 값 'a'로 초기화한다.
 28 문자열 변수 result를 선언하고, rf(str, 1, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```

29 public static String rf(String str, int index, boolean[] seen) { // index = 1, c = 'b'
30     if(index < 0) return "";
31     char c = str.charAt(index);
32     String result = rf(str, index-1, seen);

```

- 29 문자열을 반환하는 rf() 메소드의 시작점이다. 28번에서 전달한 str, 1, seen을 str, index, seen이 받는다.
 30 index가 1이므로 다음 문장을 수행한다.
 31 문자형 변수 c를 선언하고, str[1]의 값 'b'로 초기화한다.

㉔ 문자열 변수 result를 선언하고, rf(str, 0, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```
㉓ public static String rf(String str, int index, boolean[] seen) { // index = 0, c = 'a'
㉔     if(index < 0) return "";
㉕     char c = str.charAt(index);
㉖     String result = rf(str, index-1, seen);
```

㉓ 문자열을 반환하는 rf() 메소드의 시작점이다. ㉔번에서 전달한 str, 0, seen을 str, index, seen이 받는다.

㉔ index가 0이므로 다음 문장을 수행한다.

㉕ 문자형 변수 c를 선언하고, str[0]의 값 'a'로 초기화한다.

㉖ 문자열 변수 result를 선언하고, rf(str, -1, seen) 메소드를 호출한 후 반환받은 값으로 초기화한다.

```
㉗ public static String rf(String str, int index, boolean[] seen) { // index = -1
㉘     if(index < 0) return "";
        char c = str.charAt(index);
        String result = rf(str, index-1, seen);
```

㉗ 문자열을 반환하는 rf() 메소드의 시작점이다. ㉖번에서 전달한 str, -1, seen을 str, index, seen이 받는다.

㉘ index가 0보다 작으므로 빈 문자열을 반환하면서 제어를 rf(str, -1, seen)을 호출했던 ㉓번으로 옮긴다.

```
㉓ public static String rf(String str, int index, boolean[] seen) { // index = 0, c = 'a'
㉔     if(index < 0) return "";
㉕     char c = str.charAt(index);
㉖㉙ String result = rf(str, index-1, seen);
㉚     if(!seen[c]) {
㉛         seen[c] = true;
㉜         return c + result;
㉝     }
㉞     return result;
}
```

㉙ ㉓번에서 반환받은 빈 문자열로 result를 초기화한다.

result

㉚ seen[c]의 값이 거짓(false)이면 ㉚, ㉜번 문장을 수행한다. c에는 현재 'a'가 저장되어 있으며 'a'에 해당하는 아스키코드 값이 97이므로, seen[97]의 값을 확인한다. 배열 seen의 모든 요소에는 초기값으로 false가 저장되어 있으므로, 조건을 만족하여 ㉚, ㉜번 문장을 수행한다.

※ 문자가 메모리에 저장될 때는 각각의 문자에 해당하는 아스키코드 값이 저장되며, 위치 값에 사용되는 문자 'a'도 아스키코드 값으로 변환되어 사용됩니다. 즉 'a'는 'a'에 해당하는 아스키코드 값인 97로 변환됩니다.

- 알파벳 소문자의 아스키코드 값은 'a(97)' ~ 'z(122)'입니다.

- 알파벳 대문자의 아스키코드 값은 'A(65)' ~ 'Z(90)'입니다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	false	false	false	false	...	false	false	false

㉚ 논리값 true(참)를 seen[97]에 저장한다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	false	false	false	...	false	false	false

㉜ c의 값 'a'를 result의 앞쪽에 덧붙인다.

result a

㉔ result를 반환하면서 제어를 rf(str, 0, seen)을 호출했던 ㉔번으로 제어를 옮긴다.

```
㉔ public static String rf(String str, int index, boolean[] seen) { // index = 1, c = 'b'
㉕     if(index < 0) return "";
㉖     char c = str.charAt(index);
㉗㉘     String result = rf(str, index-1, seen);
㉙     if(!seen[c]) {
㉚         seen[c] = true;
㉛         return c + result;
㉜     }
㉝     return result;
㉞ }
```

㉘ ㉔번에서 반환받은 result 값으로 result를 초기화한다.

result a

㉙ seen[c]의 값이 거짓(false)이면 ㉚, ㉛번 문장을 수행한다. c에는 현재 'b'가 저장되어 있으며 'b'에 해당하는 아스키코드 값이 98이므로, seen[98]의 값을 확인한다. 배열 seen의 모든 요소에는 초기값으로 false가 저장되어 있으므로, 조건을 만족하여 ㉚, ㉛번 문장을 수행한다.

㉚ 논리값 true(참)를 seen[98]에 저장한다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	false	false	...	false	false	false

㉛ c의 값 'b'를 result의 앞쪽에 덧붙인다.

result ba

㉝ result를 반환하면서 rf(str, 1, seen)을 호출했던 ㉝번으로 제어를 옮긴다.

```
㉕ public static String rf(String str, int index, boolean[] seen) { // index = 2, c = 'a'
㉖     if(index < 0) return "";
㉗     char c = str.charAt(index);
㉘㉙     String result = rf(str, index-1, seen);
㉚     if(!seen[c]) {
㉛         seen[c] = true;
㉜         return c + result;
㉝     }
㉞     return result;
㉟ }
```

㉙ ㉘번에서 반환받은 result 값으로 result를 초기화한다.

result ba

㉚ seen[c]의 값이 거짓(false)이면 중괄호 안의 문장을 수행한다. c에는 현재 'a'가 저장되어 있으며 'a'에 해당하는 아스키코드 값이 97이므로, seen[97]의 값을 확인한다. seen[97]의 값이 참(true)이므로, 조건을 만족하지 않아 ㉚번으로 제어를 옮긴다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	false	false	...	false	false	false

㉚ result를 반환하면서 rf(str, 2, seen)을 호출했던 ㉚번으로 제어를 옮긴다.

```

21 public static String rf(String str, int index, boolean[] seen) { // index = 3, c = 'c'
22     if(index < 0) return "";
23     char c = str.charAt(index);
24 52 String result = rf(str, index-1, seen);
53     if(!seen[c]) {
54         seen[c] = true;
55         return c + result;
56     }
57     return result;
58 }

```

52 51번에서 반환받은 result 값으로 result를 초기화한다.

result ba

53 seen[c]의 값이 거짓(false)이면 54, 55번 문장을 수행한다. c에는 현재 'c'가 저장되어 있으며 'c'에 해당하는 아스키코드 값이 99이므로, seen[99]의 값을 확인한다. 배열 seen의 모든 요소에는 초기값으로 false가 저장되어 있으므로, 조건을 만족하여 54, 55번 문장을 수행한다.

54 논리값 true(참)를 seen[99]에 저장한다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	true	false	...	false	false	false

55 c의 값 'c'를 result의 앞쪽에 덧붙인다.

result cba

56 result를 반환하면서 rf(str, 3, seen)을 호출했던 57번으로 제어를 옮긴다.

```

17 public static String rf(String str, int index, boolean[] seen) { // index = 4, c = 'a'
18     if(index < 0) return "";
19     char c = str.charAt(index);
20 57 String result = rf(str, index-1, seen);
58     if(!seen[c]) {
59         seen[c] = true;
60         return c + result;
61     }
62     return result;
63 }

```

57 56번에서 반환받은 result 값으로 result를 초기화한다.

result cba

58 seen[c]의 값이 거짓(false)이면 중괄호 안의 문장을 수행한다. c에는 현재 'a'가 저장되어 있으며 'a'에 해당하는 아스키코드 값이 97이므로, seen[97]의 값을 확인한다. seen[97]의 값이 참(true)이므로, 조건을 만족하지 않아 제어를 59으로 옮긴다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	true	false	...	false	false	false

59 result를 반환하면서 rf(str, 4, seen)을 호출했던 60번으로 제어를 옮긴다.

```

13 public static String rf(String str, int index, boolean[] seen) { // index = 5, c = 'b'
14     if(index < 0) return "";
15     char c = str.charAt(index);
16 60 String result = rf(str, index-1, seen);
    61     if(!seen[c]) {
        seen[c] = true;
        return c + result;
    }
    62     return result;
    }

```

60 59번에서 반환받은 result 값으로 result를 초기화한다.

result

cba

61 seen[c]의 값이 거짓(false)이면 중괄호 안의 문장을 수행한다. c에는 현재 'b'가 저장되어 있으며 'b'에 해당하는 아스키코드 값이 98이므로, seen[98]의 값을 확인한다. seen[98]의 값이 참(true)이므로, 조건을 만족하지 않아 62번으로 제어를 옮긴다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	true	false	...	false	false	false

62 result를 반환하면서 rf(str, 5, seen)을 호출했던 63번으로 제어를 옮긴다.

```

9 public static String rf(String str, int index, boolean[] seen) { // index = 6, c = 'c'
10     if(index < 0) return "";
11     char c = str.charAt(index);
12 63 String result = rf(str, index-1, seen);
    64     if(!seen[c]) {
        seen[c] = true;
        return c + result;
    }
    65     return result;
    }

```

63 62번에서 반환받은 result 값으로 result를 초기화한다.

result

cba

64 seen[c]의 값이 거짓(false)이면 중괄호 안의 문장을 수행한다. c에는 현재 'c'가 저장되어 있으며 'c'에 해당하는 아스키코드 값이 99이므로, seen[99]의 값을 확인한다. seen[99]의 값이 참(true)이므로, 조건을 만족하지 않아 65번으로 제어를 옮긴다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	true	false	...	false	false	false

65 result를 반환하면서 rf(str, 6, seen)을 호출했던 66번으로 제어를 옮긴다.

```

5 public static String rf(String str, int index, boolean[] seen) { // index = 7, c = 'd'
6     if(index < 0) return "";
7     char c = str.charAt(index);
8 66 String result = rf(str, index-1, seen);
9     if(!seen[c]) {
10         seen[c] = true;
11         return c + result;
12     }
13     return result;
14 }

```

66 65번에서 반환받은 result 값으로 result를 초기화한다.

result cba

67 seen[c]의 값이 거짓(false)이면 68, 69번 문장을 수행한다. c에는 현재 'd'가 저장되어 있으며 'd'에 해당하는 아스키코드 값이 100이므로, seen[100]의 값을 확인한다. 배열 seen의 모든 요소에는 초기값으로 false가 저장되어 있으므로, 조건을 만족하여 68, 69번 문장을 수행한다.

68 논리값 true(참)를 seen[100]에 저장한다.

	[0]	[1]	[2]	...	[97]	[98]	[99]	[100]	...	[253]	[254]	[255]
seen	false	false	false	...	true	true	true	true	...	false	false	false

69 c의 값 'c'를 result의 앞쪽에 덧붙인다.

result dcba

70 result를 반환하면서 rf(str, 7, seen)을 호출했던 7번으로 제어를 옮긴다.

```

public static void main(String[] args) {
1     String str = "abacabcd";
2     int len = str.length( );
3     boolean[] seen = new boolean[256];
4 71 System.out.print(rf(str, len-1, seen));
5 }

```

71 70번에서 반환받은 result의 값을 출력한다.

결과 dcba

[문제 9]

21

[해설]

```

#include <stdio.h>

int main( ) {
1     int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
2     int* parr[2] = {arr[1], arr[2]};
3     printf("%d", parr[1][1] + *(parr[1]+2) + **parr);
4 }

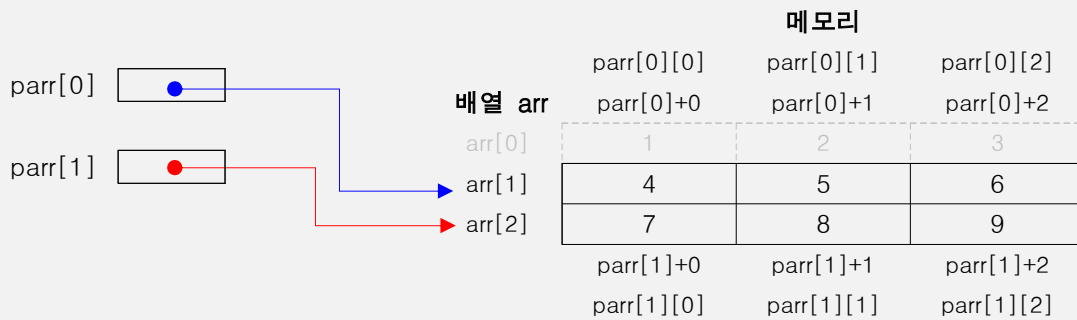
```

❶ 3행 3열의 크기를 갖는 정수형 배열 arr을 선언하고, 초기화한다.

		arr[0][0]	arr[0][1]	arr[0][2]
배열 arr	arr[0]	1	2	3
	arr[1]	4	5	6
	arr[2]	7	8	9
		arr[2][0]	arr[2][1]	arr[2][2]

❷ 2개의 요소를 갖는 정수형 포인터 배열 parr을 선언하고, parr[0]에 배열 arr의 두 번째 행의 시작 주소를, parr[1]에 배열 arr의 세 번째 행의 시작 주소를 저장한다.

※ 2차원 배열에서 배열명은 첫 번째 행의 시작 주소를 가리킵니다. 즉 arr은 첫 번째 행의 시작 주소를 가리키고, arr[1]은 두 번째 행의 시작 주소를, arr[2]는 세 번째 행의 시작 주소를 가리킵니다.



❸ printf("%d", parr[1][1] + *(parr[1]+2) + **parr);

㉠

㉡

㉢

- ㉠ : parr[1]은 arr 배열의 세 번째 행의 시작 주소를 가리키므로, parr[1][1]은 3행의 두 번째 열의 값 8입니다.
- ㉡ : parr[1]은 arr 배열의 세 번째 행의 시작 주소를 가리키고, 여기에 주소 2를 더하면 3행의 세 번째 열이므로 값은 9입니다.
- ㉢ : *parr은 parr[0]을 의미하며, **parr은 parr[0][0], 즉 arr[1][0]인 2행의 첫 번째 열이므로 값은 4입니다.
- 8+9+4의 결과를 정수형으로 출력합니다.

결과 21

[문제 10]

- VALUES
- SELECT
- FROM
- SET

※ 답안 작성 시 주의 사항 : SQL에 사용되는 예약어, 필드명, 변수명 등은 대소문자를 구분하지 않기 때문에 소문자로 작성해도 정답으로 인정됩니다.

[해설]

• <SQL 1> 문

INSERT INTO 학부(학번, 이름, 주소, 나이)

<학부> 테이블의 '학번', '이름', '주소', '나이' 필드에 삽입한다.

VALUES (240912, '최재균', '서울', 20);

240912를 '학번' 필드에, '최재균'을 '이름' 필드에, '서울'을 '주소' 필드에, 20을 '나이' 필드에 삽입한다.

• <SQL 2> 문

INSERT INTO 학생(학번, 이름, 나이, 학과)

<학생> 테이블의 '학번', '이름', '나이', '학과' 필드에 삽입한다.

SELECT 학번, 이름, 나이, '컴퓨터공학'

'학번', '이름', '나이' 필드는 조회하여 삽입하고, 네 번째 필드에는 '컴퓨터공학' 값을 직접 삽입한다.

FROM 학부

<학부> 테이블을 대상으로 조회한다.

WHERE 이름 = '최재균';

'이름'이 '최재균'인 튜플만을 대상으로 한다.

• <SQL 3> 문

SELECT *

모든 필드를 조회한다.

FROM 학생

<학생> 테이블을 대상으로 조회한다.

• <SQL 4> 문

UPDATE 학생

<학생> 테이블을 갱신한다.

SET 학과 = '휴학'

'학과'를 "휴학"으로 갱신한다.

WHERE 학번 = 240912;

'학번'이 240912인 튜플만을 대상으로 한다.

[문제 11]

① 5

② 4

[문제 12]

※ 다음 중 하나를 쓰면 됩니다.

IPsec, IP Security

[문제 13]

※ 다음 중 하나를 쓰면 됩니다.

AES, Advanced Encryption Standard

[문제 14]

※ 각 문항별로 다음 중 하나를 쓰면 됩니다.

① 가상 회선, 가상 회선 방식, VC, Virtual Circuit

② 데이터그램, 데이터그램 방식, Datagram

[문제 15]

㉠

[문제 16]

[해설]

SRT는 현재 실행중인 프로세스의 남은 시간과 준비상태 큐에 새로 도착한 프로세스의 실행 시간을 비교하여 가장 짧은 실행 시간을 요구하는 프로세스에게 CPU를 할당하는 기법으로, 실행이 마무리되지 못한 경우 준비상태 큐에 재배치하여 차례를 기다리므로 다음과 같이 표시할 수 있다.

프로세스	A	B	C	D
도착 시간	0	1	2	3
실행 시간	8	4	9	5

진행 시간 →	0	1	⑤	⑩	⑰	⑳
프로세스	A	B	D	A	C	
실행 시간	1	4	5	7	9	
남은 시간	7	0	0	0	0	

※ 원문자는 프로세스가 완료됨을 표시한 것입니다.

※ 대기 시간은 '완료 시간 - 도착 시간 - 실행 시간'으로 구할 수 있으며, 프로세스별 대기 시간은 다음과 같습니다.

- A : $17 - 0 - 8 = 9$
- B : $5 - 1 - 4 = 0$
- C : $26 - 2 - 9 = 15$
- D : $10 - 3 - 5 = 2$

※ 평균 대기 시간 = 전체 대기 시간 / 프로세스의 수 = $26 / 4 = 6.5$

[문제 17]

Iterator

[문제 18]

㉞

[문제 19]

20

[해설]

```
#include <stdio.h>
struct node {                // node 구조체를 정의합니다.
    int data;                // 정수형 변수 data를 선언합니다.
    struct node *Next;       // node 구조체의 포인터 변수 Next를 선언합니다.
};
```

struct node	
int data (4Byte)	struct node *Next (4Byte)
데이터를 저장할 멤버	다음 노드의 주소를 저장할 포인터

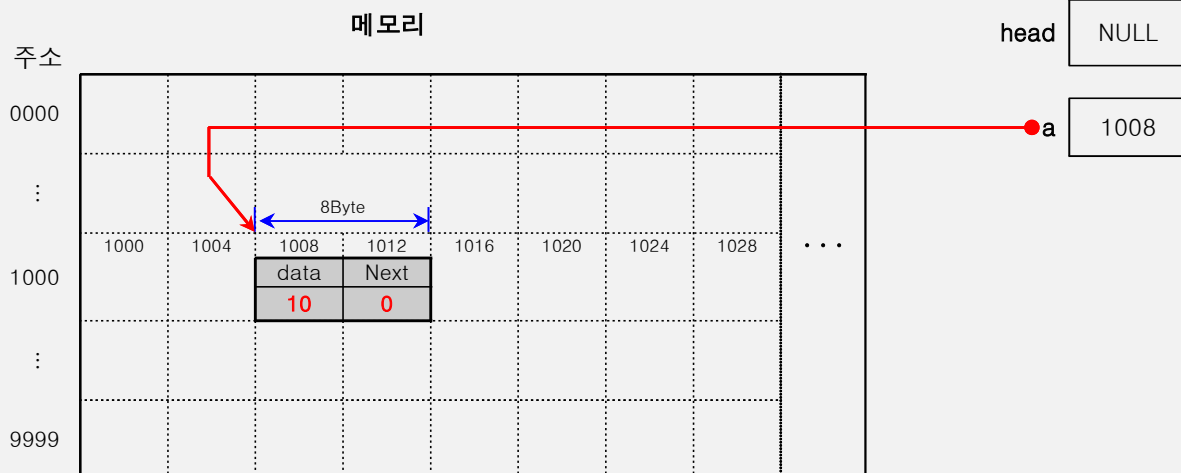
```
int main( ) {
    ❶ struct node *head = NULL;
    ❷ struct node a = {10, 0};
    ❸ struct node b = {20, 0};
    ❹ struct node c = {30, 0};
    ❺ head = &a;
    ❻ a.Next = &b;
    ❼ b.Next = &c;
    ❽ printf("%d", head -> Next -> data);
}
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

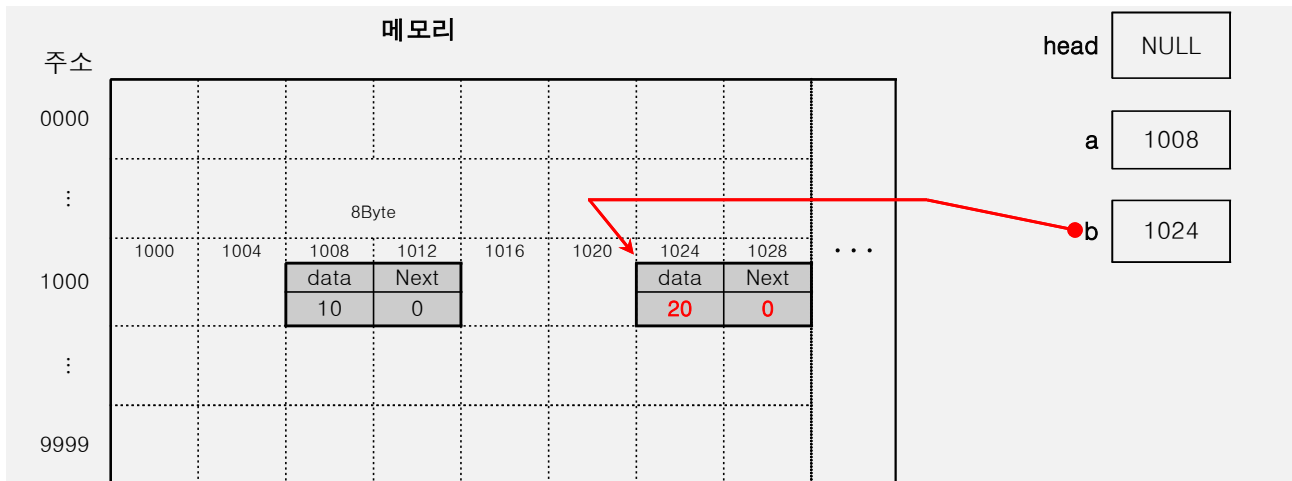
- ❶ node 자료형 포인터 변수 head를 선언하고, NULL로 초기화한다. 포인터 변수의 크기는 4Byte이므로 메모리의 어딘가에 4Byte 크기의 공간이 할당된다. 포인터 변수 head의 용도는 리스트 구조에서 첫 번째 노드의 주소를 저장하는 것이다.

head NULL

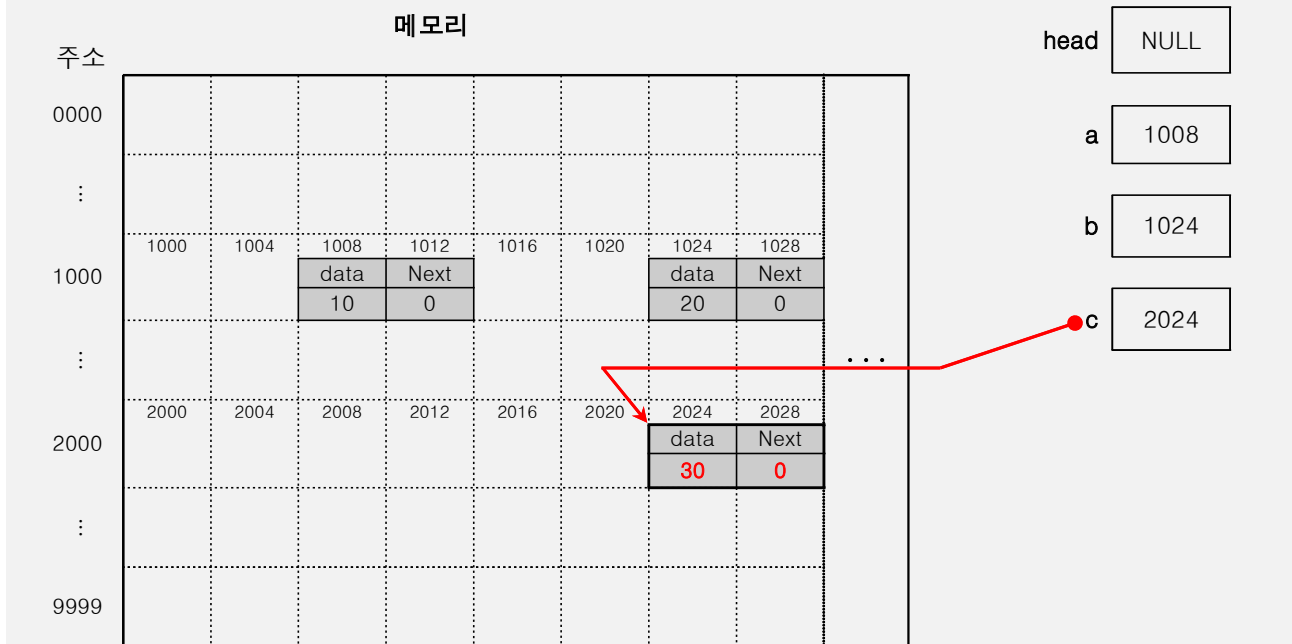
- ❷ node 자료형 변수 a를 선언하고, node의 data에 10을 node의 Next에 0을 저장한다. a가 가리키고 있는 1008 번지는 node 구조체의 크기만큼 할당된 공간이므로 a는 1008 번지 이후의 8Byte를 의미한다. (이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했습니다.)



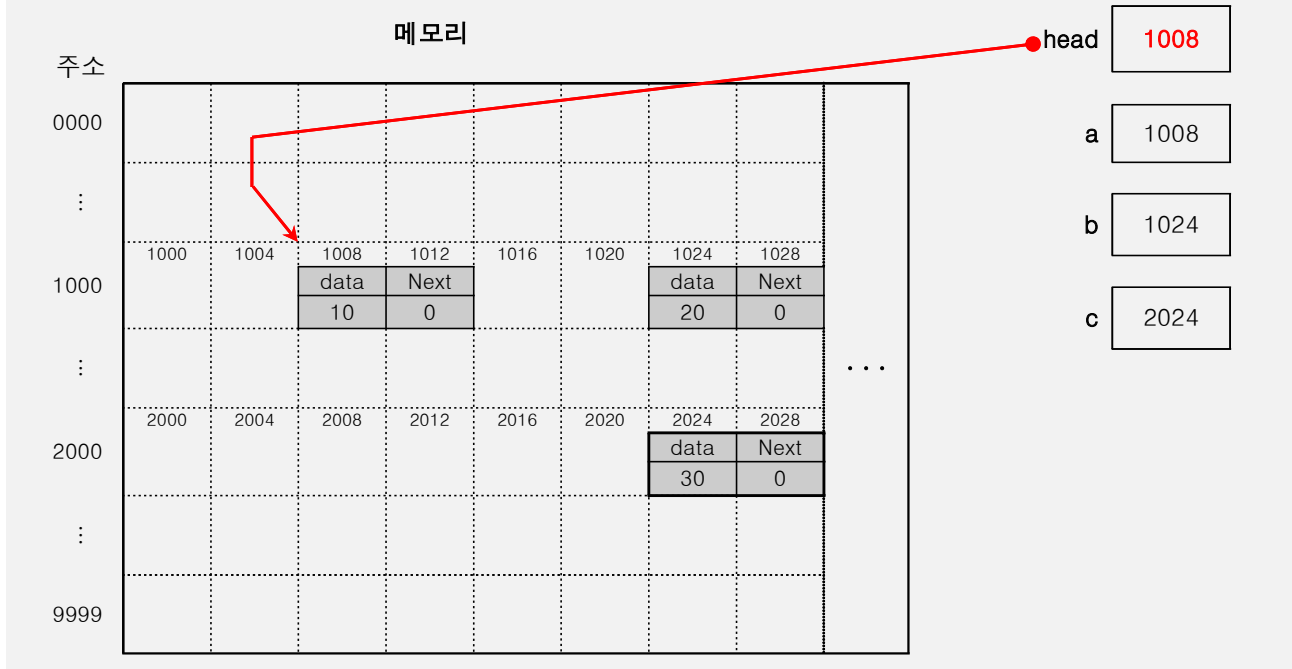
- ❸ node 자료형 변수 b를 선언하고, node의 data에 20을 node의 Next에 0을 저장한다.



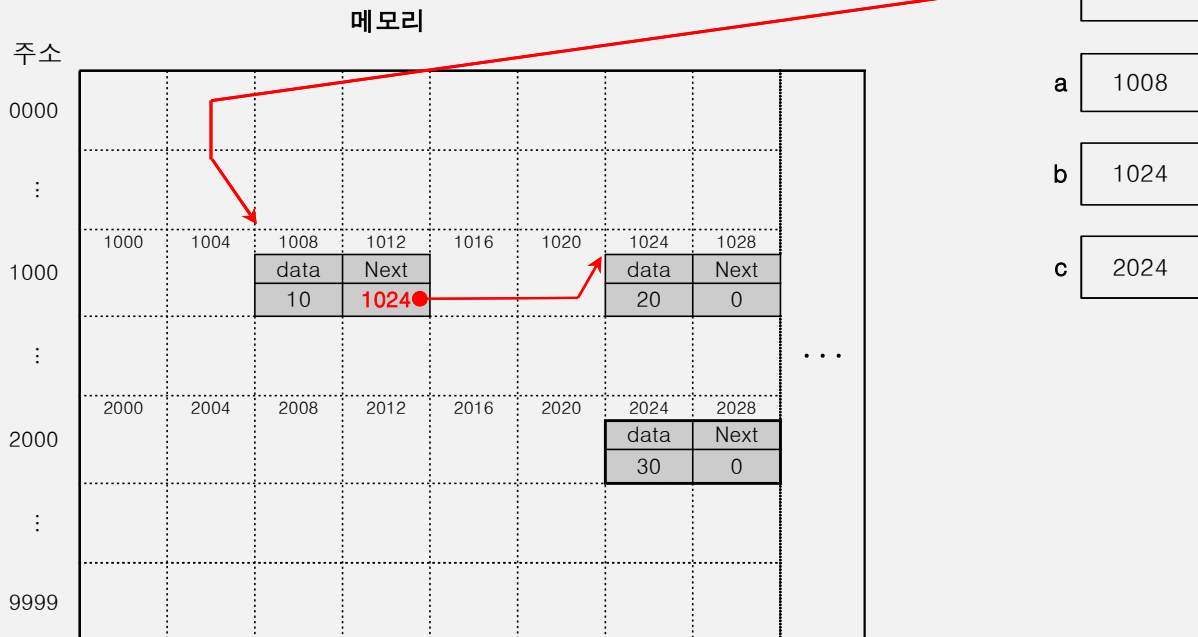
④ node 자료형 변수 c를 선언하고, node의 data에 30을 node의 Next에 0을 저장한다.



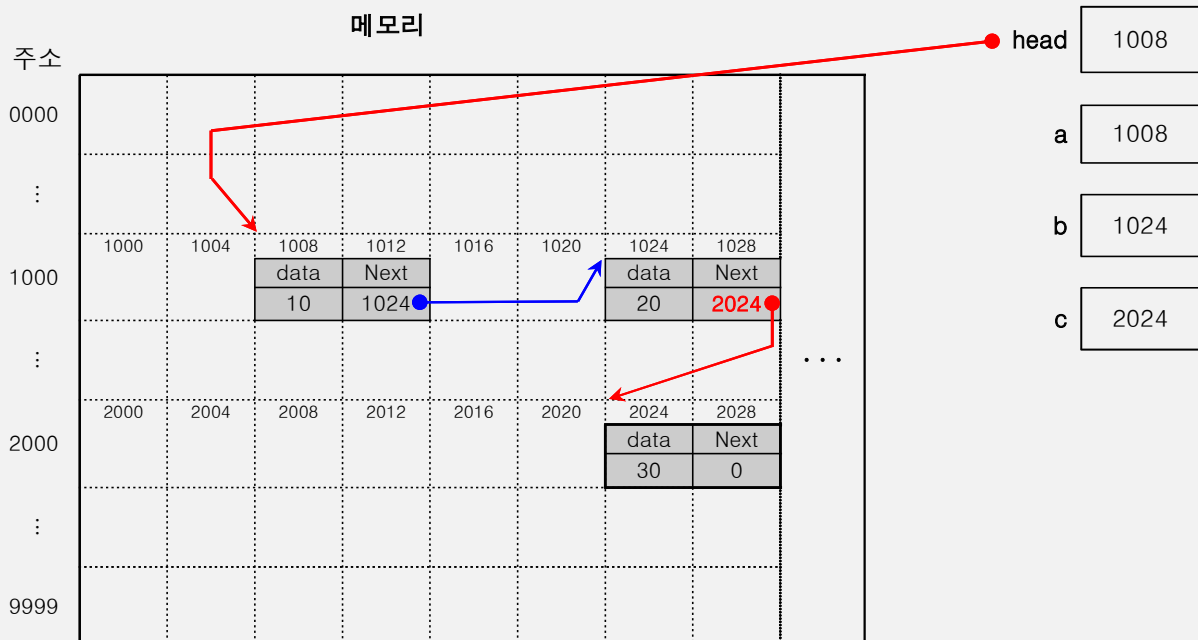
⑤ a의 주소를 head에 저장한다. head의 용도는 리스트 구조에서 첫 번째 노드의 주소를 저장하는 것이므로, 이제 a가 리스트 구조의 첫 번째 노드가 되는 것이다.



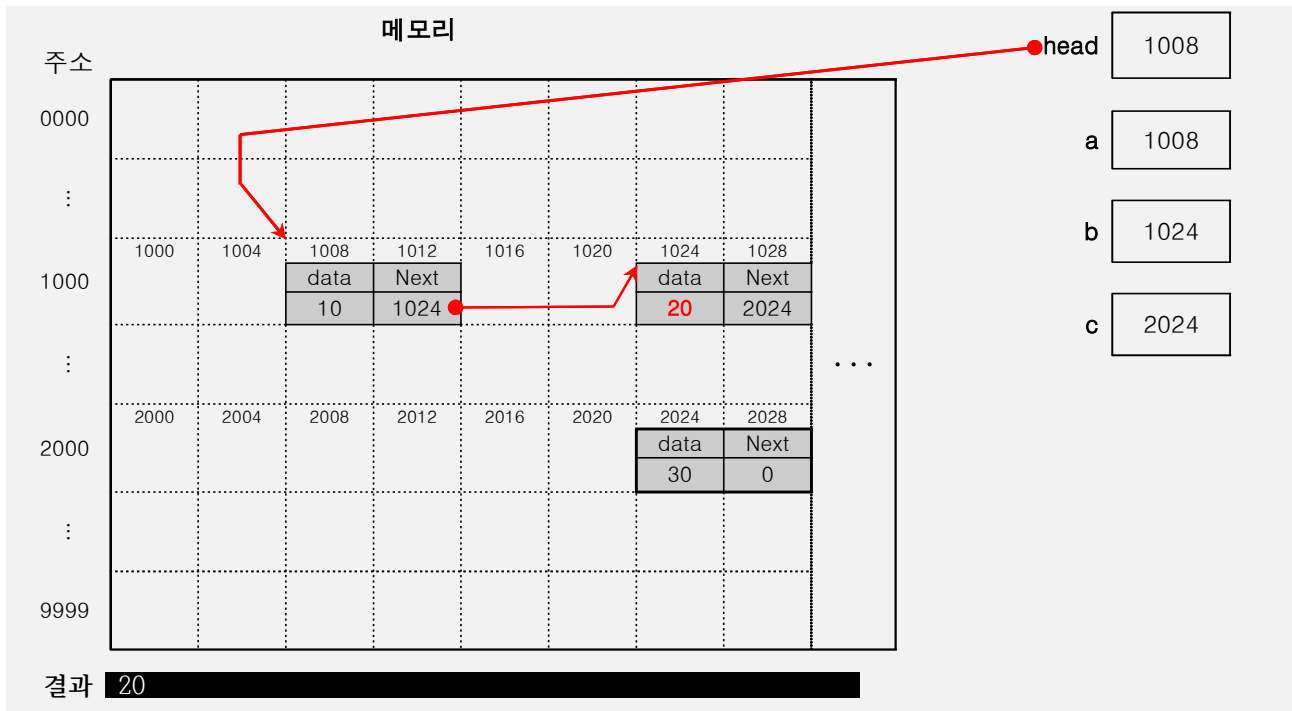
- ⑥ b의 주소를 a의 Next에 저장한다. a가 가리키고 있는 1008 번지는 node 구조체의 크기만큼 할당된 공간이므로 a의 Next는 1012 번지 이후의 4Byte를 의미하며, 그곳에 b의 주소인 1024를 저장한다. Next는 다음 노드의 주소를 지정하는 역할을 하므로, 이제 a의 다음 노드는 b가 된다.



- ⑦ c의 주소를 b의 Next에 저장한다. Next는 다음 노드의 주소를 지정하는 역할을 하므로, 이제 b의 다음 노드는 c가 된다.



- ⑧ head가 가리키는 곳의 Next가 가리키는 곳의 data를 출력한다.



[문제 20]

A → D → C → F

[해설]

RIP은 홉 카운트(Hop Count), 즉 라우터의 수를 기준으로 가장 적은 수의 라우터를 거쳐가는 경로를 최적 경로로 선택합니다. 문제의 그림에서 확인할 수 있는 A에서 F까지의 경로는 다음과 같습니다.

- A → B → D → C → E → F
- A → B → D → C → F
- A → D → C → E → F
- A → D → C → F

∴ 가장 적은 수의 라우터를 거쳐가는 A → D → C → F가 최적 경로입니다.